Réseau de transport : fonctionnement et mise à jour

Ce document explique comment est construit le réseau de transport affiché sur Refuges des Alpes, d'où viennent les données, et comment le tenir à jour. Il ne demande pas de connaître le code en détail.

En bref

- Le réseau est **calculé à l'avance sur un ordinateur**, puis enregistré dans des fichiers fixes du site. Le site en ligne ne fait que lire ces fichiers : aucun calcul, aucune IA, aucun coût par visiteur.
- Les données viennent surtout des **flux officiels GTFS** publiés sur transport.data.gouv.fr (cars régionaux, TER, réseaux urbains, navettes de stations).
- Les navettes absentes de l'open data sont **saisies à la main** dans un fichier, avec leur source et leurs dates.
- Une seule commande met tout à jour : `npm run update-transports`. Une autre vérifie s'il y a du nouveau : `npm run check-transports`.

Chiffres clés

Indicateur	Ancien réseau	Réseau actuel
Refuges français desservis à moins de 5 km	43 %	71 %
Taille du fichier chargé à l'ouverture	40 Mo	3,6 Mo (0,83 Mo compressé)
Mention « certifié »	oui, sans fondement	retirée

D'où viennent les données

1. Les flux officiels (la grande majorité)

Le catalogue des sources se trouve dans `scripts/transport_v2/sources.cjs`. Il liste environ 60 jeux de données de transport.data.gouv.fr, rangés en trois familles :

- **Cars régionaux et trains** : Cars Région Isère, Savoie, Haute-Savoie, ZOU !, TER, SNCF (seulement les trains régionaux, sans TGV ni Intercités).
- **Réseaux urbains et de vallée** : Chamonix Bus, Altigo, Tougo, TAG, Sibra, Lignes d'Azur…
- **Navettes de stations** : Alpe d'Huez, Val d'Isère, Tignes, Méribus, Courchevel, Col'ette…

Chaque entrée peut être désactivée (`actif: false`) sans être supprimée, par exemple pour FlixBus ou Eurostar.

2. Les navettes hors open data

Certaines navettes estivales ne sont publiées nulle part en format ouvert, seulement dans des PDF ou sur des sites d'office de tourisme. Elles sont décrites à la main dans `scripts/transport_v2/navettes_hors_open_data.json` . Aujourd'hui, 8 navettes :

- Haute Maurienne Vanoise, lignes 2, 3 et 4 ;
- Peisey, circuits B et C vers Rosuel ;
- Navette du Valgaudemar ;
- Morzine, ligne E vers le lac des Mines d'Or ;
- Queyras « Andiamo ! » vers le col Agnel (relevé partiel).

Chaque navette indique sa source (lien du document), la date de consultation, la saison couverte, les arrêts et les horaires. Sur la carte, elles portent le badge « Hors open data » et leur source est affichée.

3. Les archives des flux saisonniers

Quelques flux de stations se vident entre deux saisons : Val d'Isère, Tignes, Alpe d'Huez, Valmorel. Pour ne pas perdre ces lignes, la dernière bonne version de chaque flux est gardée dans `scripts/transport_v2/archives/` (environ 1,6 Mo). Quand un flux officiel est vide, c'est l'archive qui est utilisée, et la ligne est marquée « Hors saison » sur le site.

Comment le réseau est construit

La construction suit quatre étapes, lancées automatiquement par `npm run update-transports` .

1. **Collecte** (`1_collecter_sources.cjs`) : télécharge le catalogue officiel, les arrêts autour des refuges et la commune de chaque refuge. Ces fichiers de travail vont dans `imports/transport_v2/` , qui n'est pas versionné.
2. **Tracés des navettes manuelles** (`navettes_tracer.cjs`) : calcule le trajet routier de chaque navette saisie à la main, grâce à OpenStreetMap. Il ne recalcule que les navettes dont les arrêts ont changé.

3. **Construction** (`3_construire_reseau.cjs`) : le cœur du travail, détaillé ci-dessous.
4. **Rapport de couverture** (`2_analyser_couverture.cjs`) : liste les refuges desservis ou non, dans `docs/transport/rapport_couverture_v2.md` .

Ce que fait l'étape de construction

- **Lecture des flux GTFS** : arrêts, lignes, horaires, jours de circulation et tracés.
- **Tri des lignes** : une ligne est gardée si l'un de ses arrêts est à moins de 5 km d'un refuge, et pas plus de 1,5 km plus loin que le meilleur arrêt officiel de ce refuge. Cela évite d'afficher toutes les lignes urbaines d'une ville voisine.
- **Retrait des circuits scolaires** : une ligne qui ne roule qu'en semaine, avec au plus deux départs par sens et un terminus à un collège ou un lycée, est écartée.
- **Types de jours** : les horaires sont regroupés en Semaine, Samedi et Dimanche, d'après les 35 prochains jours.
- **Saisons** : la période de circulation est calculée. Une ligne qui ne roule plus à aucune date future est marquée « Hors saison ».
- **Doublons** : un même train présent dans deux flux (TER et SNCF par exemple) n'est gardé qu'une fois.
- **Ajout des navettes manuelles et des archives.**

Ce que le site charge

La construction produit des fichiers dans `public/` :

Fichier	Rôle	Quand il est chargé
`transports_v2.json`	Tracés et noms de toutes les lignes, pour la carte	À l'ouverture de la couche transport
`transports_v2/lignes/<id>.json`	Horaires détaillés d'une ligne	Au clic sur la ligne
`transports_v2_search_index.json`	Index pour la barre de recherche	À la recherche

Côté code du site, l'adresse des fichiers est définie dans `src/features/transport/transportSource.js` . L'affichage de la carte est dans `TransportLayer.jsx` et le panneau des horaires dans `TransportSidebar.jsx` . Le service worker (`public/sw.js`) va chercher ces fichiers sur le réseau en priorité, pour que les visiteurs aient toujours la dernière version.

Comment mettre à jour

Vérifier s'il y a du nouveau

```
npm run check-transports
```

Cette commande ne modifie rien. Elle affiche :

- les flux officiels republiés depuis la dernière construction ;
- les navettes manuelles dont la saison relevée est terminée (il faut alors chercher le document de la saison suivante).

Tout reconstruire

```
npm run update-transports
```

Compter plusieurs minutes. La commande a besoin d'internet et de beaucoup de mémoire. À la fin, deux comptes rendus sont écrits :

- `docs/transport/construction_reseau.md` : lignes gardées, écartées, et pourquoi ;
- `docs/transport/rapport_couverture_v2.md` : refuges desservis ou non.

Ensuite :

1. lancer le site en local (`npm run dev`) et vérifier quelques lignes sur la carte ;
2. relire rapidement les deux comptes rendus, surtout si la couverture a beaucoup baissé ;
3. faire un commit des fichiers modifiés dans `public/` et `scripts/transport_v2/archives/` , puis pousser.

Rythme conseillé

- **Avant chaque saison** (début juin et début décembre) : c'est là que les navettes et les stations publient leurs nouveaux horaires.
- **Une fois par mois** environ, lancer `check-transports` pour voir si un flux important a changé.

Ajouter ou corriger une navette manuelle

1. Trouver le document officiel (PDF ou page de l'office de tourisme, de la commune ou du réseau).

2. Ouvrir `scripts/transport_v2/navettes_hors_open_data.json` et ajouter ou modifier une entrée : nom, source (lien et date de consultation), saison (début et fin), arrêts (nom et coordonnées) et grilles horaires par sens.
3. Lancer `npm run update-transports`. Le tracé routier est calculé automatiquement, et le fichier est vérifié : une erreur de format est signalée clairement.
4. Vérifier la navette sur la carte, puis faire un commit.

Le format complet est décrit dans `docs/transport/navettes_hors_open_data.md`.

Ajouter ou retirer un réseau officiel

1. Trouver le jeu de données sur transport.data.gouv.fr et noter son identifiant (dans l'adresse de la page).
2. L'ajouter dans `scripts/transport_v2/sources.cjs` avec un nom de réseau (et `navette: true` si c'est une navette de station).
3. Pour retirer un réseau sans le perdre, mettre `actif: false`.
4. Lancer `npm run update-transports`.

Limites connues

- Les données officielles ne sont pas parfaites : certains réseaux publient en retard, ou vident leur flux hors saison. Les archives limitent ce problème mais ne le suppriment pas.
- Les navettes manuelles doivent être **ressaisies à chaque saison**. C'est le seul travail vraiment manuel.
- Quelques anciennes lignes n'ont pas été reprises : La Bérarde (route détruite en 2024), Galibier, Champagny-le-Haut, Giffre été (aucune source trouvée), Altigo S17 et S18 (circuits scolaires).
- Le tri des circuits scolaires repose sur une règle simple : il peut rarement écarter une vraie ligne. Si c'est le cas, on peut forcer la ligne avec un filtre `garderLigne` dans `sources.cjs`.
- Les tracés des navettes manuelles suivent la route la plus probable, pas forcément le trajet exact du car.